BajriX Full-Stack Engineering Challenge

Build a Multi-Seller Product Marketplace

Time window: 24 hours
Expected effort: 6 - 8 hours
Preferred stack: React, Java/Spring Boot, PostgreSQL

You may use **AI tools, documentation, Stack Overflow, code assistants, libraries, templates, or any other resources** you normally use while building software.

Using AI is completely allowed.

However, shortlisted candidates will have a technical discussion based on their submission and should be able to explain, debug, modify, and defend the important parts of their implementation.

Background

BajriX is building a marketplace for construction and home-building products.

A product in the BajriX catalogue may be offered by multiple local sellers.

For example:

UltraTech PPC Cement 50 kg

could potentially be available from:

- Seller A at ₹390/bag
- Seller B at ₹385/bag
- Seller C at ₹405/bag

Each seller may have different:

- price;
- available stock;
- minimum order quantity;
- availability.

The platform should allow buyers to discover products and compare available seller offerings.

Sellers should be able to manage the products they sell.

Your Task

Build a small working version of this marketplace.

At minimum, the system should support:

Buyer Experience

A buyer should be able to browse products and understand which sellers currently offer them.

The buyer should have reasonable ways to find relevant products.

What information you display, how products and seller offerings are presented, and how search/filtering works is up to you.

Seller Experience

A seller should be able to view and manage their own product listings.

A seller may need to:

- add a product to their catalogue;
- change price;
- change stock;
- change minimum order quantity;
- stop selling a product.

You do not need to build production-grade authentication.

You may use a mocked user, seller selector, request header, hard-coded session, or another reasonable mechanism.

However, your backend should be designed so that one seller cannot modify another seller's data.

Product and Listing Model

There is an important distinction between:

Product

and

Seller Listing

For example:

> UltraTech PPC Cement 50 kg

is a product.

The fact that:

> Shree Traders sells it for ₹390/bag with 500 bags in stock

is a seller listing.

Multiple sellers may list the same product.

Your database design should represent this distinction appropriately.

Beyond this, you are free to design the entities and relationships you believe are required.

Seller Status

Not every seller registered on the platform should necessarily be visible to buyers.

Assume sellers may have statuses such as:

`APPROVED`

`PENDING`

`REJECTED`

Decide how seller status should affect different parts of your application.

Business Behaviour

Your system should handle obvious invalid or inconsistent situations appropriately.

Examples may include:

- invalid prices;
- invalid stock;
- invalid quantities;
- duplicate seller listings;
- unavailable products;
- changes happening to existing listings.

We intentionally haven't specified every business rule.

Make reasonable decisions and document anything important.

Search and Discovery

Assume buyers need to find products efficiently.

You may support search, filters, sorting, pagination or other discovery mechanisms you consider useful.

Assume that a production version of the system may eventually contain roughly:

- 1,000,000 products;
- 100,000 sellers;
- 10,000,000 seller listings.

You are **not expected to build infrastructure for this scale**.

However, your application should not rely on approaches that would obviously stop working once the data becomes large.

Backend

Build a REST API using Java and Spring Boot.

We are interested in how you approach:

- API structure;
- validation;
- error handling;
- business logic;
- database access;
- authorization boundaries;
- maintainability.

There is no required endpoint specification.

Design the APIs you believe the frontend needs.

Frontend

Build the frontend using React.

The application should provide a clear and usable experience for the workflows you choose to implement.

We will consider both the quality of the frontend implementation and the overall user experience.

You are free to decide how information should be presented, how users should navigate the application, and what interactions or states are important.

The interface does not need to be visually elaborate or production-ready, but it should feel thoughtfully designed and reasonably polished.

Use any UI library, CSS framework, component library, or design system you prefer.

Be prepared to explain the frontend and UX decisions you made during the discussion round.

Database

Use PostgreSQL.

Include enough sample data for someone reviewing the project to understand and test the system.

Your sample data should preferably demonstrate situations such as:

- multiple sellers selling the same product;
- different seller prices;
- different stock levels;
- sellers in different statuses.

Use migrations, SQL scripts, application initialization, or another reproducible mechanism of your choice.

Concurrency

Assume it is possible that two people or processes may update the same seller listing at approximately the same time.

You are not required to build a sophisticated distributed solution.

Consider whether this affects your implementation.

Be prepared to discuss your approach.

Testing

Add whatever automated tests you believe provide meaningful value.

We deliberately do not specify:

- the type of tests;
- the number of tests;
- required coverage.

Part of the challenge is deciding what deserves testing.

What We Are NOT Asking You to Build

You do not need to implement:

- payment processing;
- checkout;
- Request for Quote workflows;
- real production authentication;
- email or SMS;
- mobile applications;
- cloud deployment;
- pixel-perfect UI.

Avoid spending significant time on functionality unrelated to the core problem.

Submission

Submit a Git repository containing the complete project.

Include a `README.md` explaining:

1. how to run the application;
2. important architectural decisions;
3. assumptions you made;
4. anything you intentionally did not implement;
5. what you would improve with more time;
6. anything you would reconsider if the platform reached millions of products/listings.

You may use Docker or Docker Compose, but it is not mandatory.

AI Usage

Include a short file named:

`AI_USAGE.md`

Mention:

- which AI/code-generation tools you used;
- what you primarily used them for;
- one example where you changed, rejected, or corrected something suggested by an AI tool.

There is no penalty for using AI extensively.

Evaluation

There is intentionally **no single correct solution**.

We will evaluate the submission based on the quality of engineering decisions, correctness, maintainability, understanding of the problem, and how well you can explain your implementation.

Shortlisted candidates will have a technical discussion based directly on their submitted project.

During that discussion, you may be asked to:

- explain parts of your code;
- justify your database model;
- modify an existing feature;
- debug an issue;
- handle a changed requirement;
- discuss scalability;
- explain something generated with AI.

You should therefore understand everything you submit.